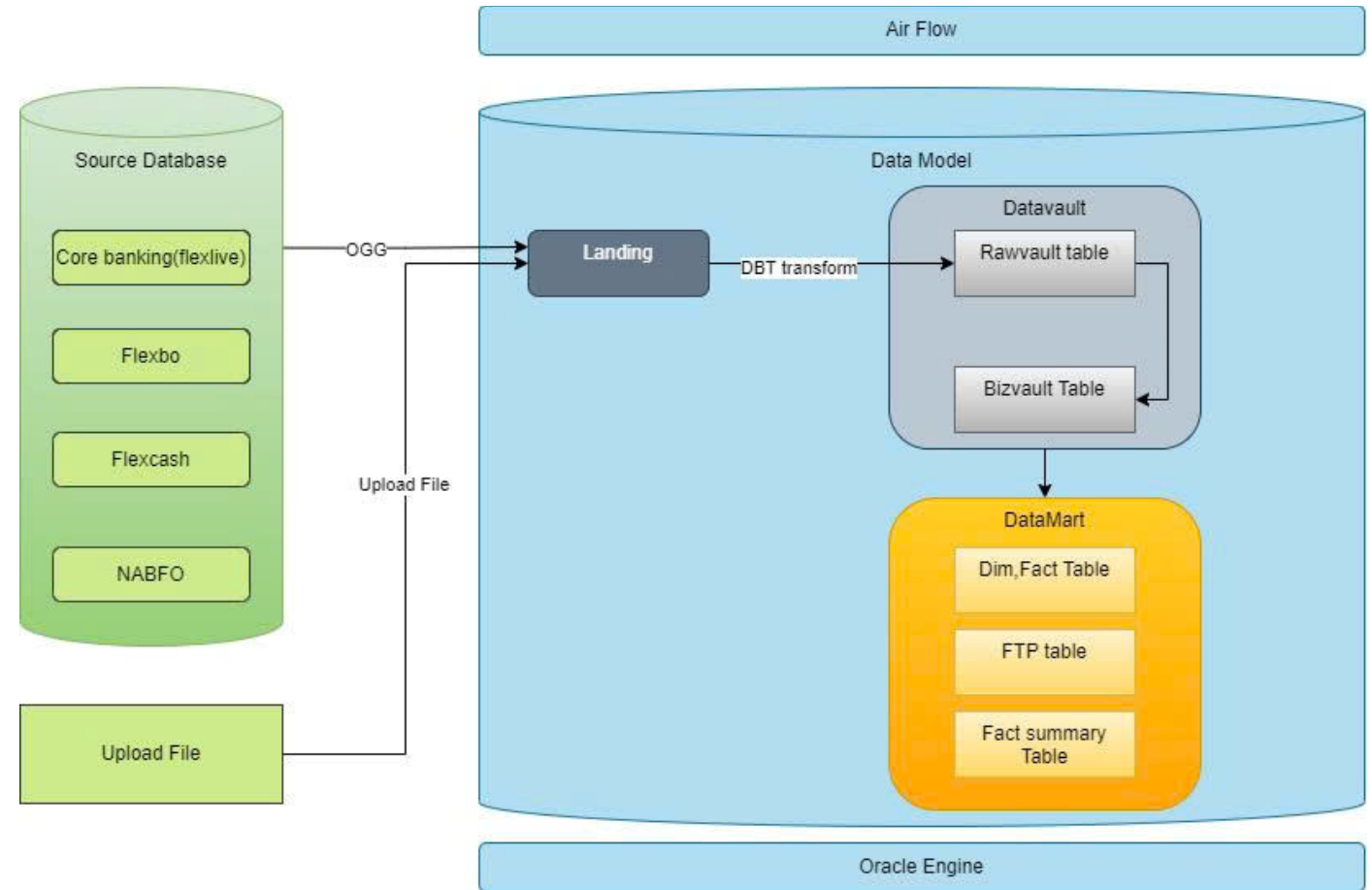


# 1. Giới thiệu kiến trúc tổng thể Data Model

- Dữ liệu từ source được đồng bộ vào vùng Landing bằng OGG hoặc upload file.
- Data Vault gồm 2 phần:
  - Raw Vault: Vùng dữ liệu Mô hình hóa thông qua kiến trúc Data Vault 2.0
  - Biz Vault: Là vùng xử lý các yêu cầu Logic nghiệp vụ phức tạp, có thể tái sử dụng tại nhiều nơi.
- Data Mart là vùng dữ liệu chuẩn hóa và làm giàu để phục vụ nhu cầu phân tích, báo cáo.
- Airflow: Công cụ điều phối luồng xử lý dữ liệu.
- Toàn bộ việc xử lý dữ liệu diễn ra trong Oracle Engine
- DBT: công cụ transform tự động cho raw vault.



## 2. Chi tiết thiết kế hệ thống

### Phân vùng Landing

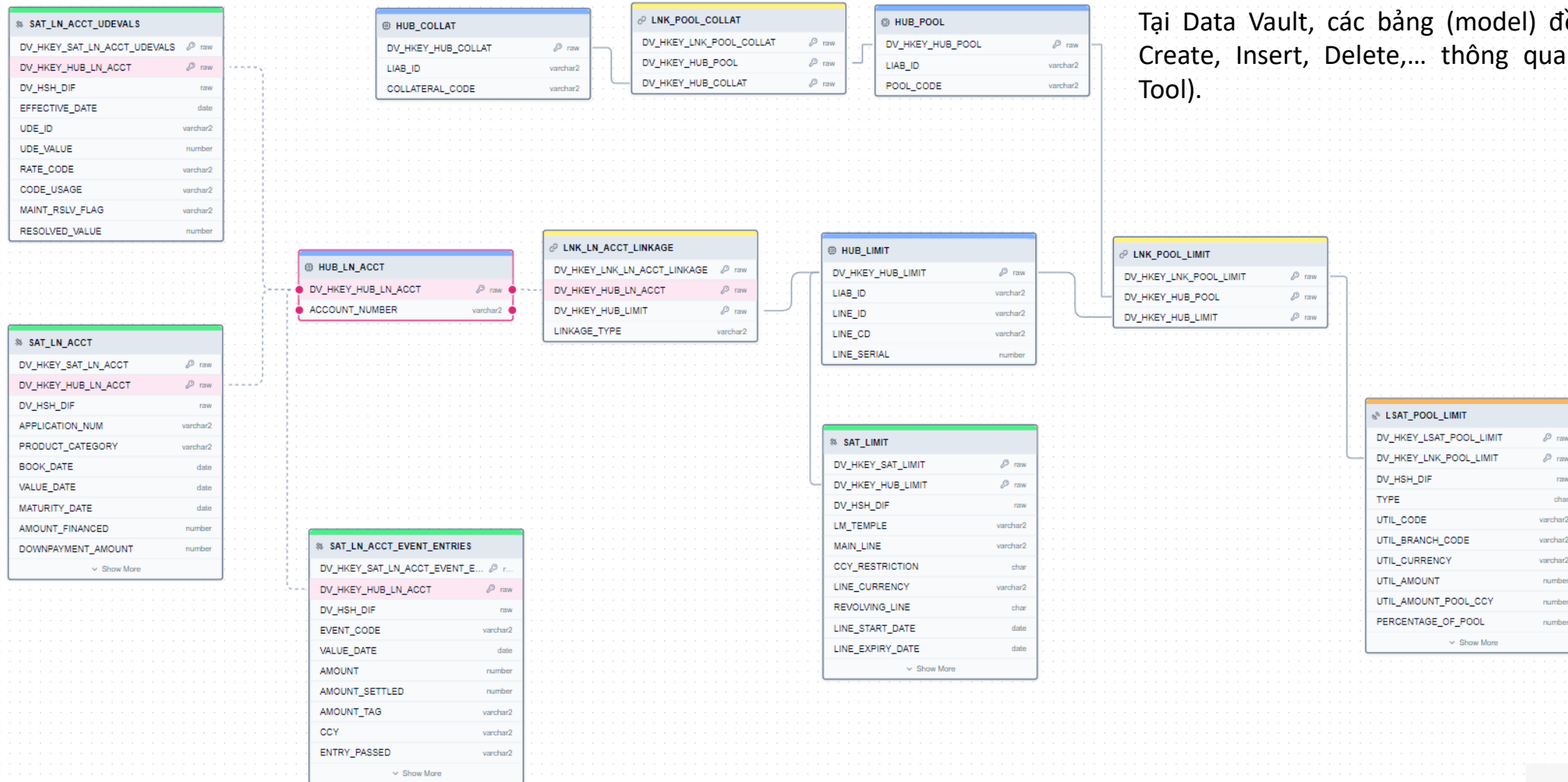
- Landing là nơi tiếp nhận dữ liệu thô từ các hệ thống nguồn mà không có bất kỳ biến đổi nào.
- Dữ liệu trong Landing Zone được lưu trữ tạm thời, phục vụ cho việc tải vào vùng DataVault.
- Bảng Landing được bổ sung các cột hệ thống hỗ trợ truy vết dữ liệu:

| Tên cột   | Kiểu dữ liệu  | Mô tả                                                                            |
|-----------|---------------|----------------------------------------------------------------------------------|
| OP_TYPE   | varchar2(255) | Loại DML (ví dụ: INSERT, UPDATE, DELETE) từ hệ thống nguồn.                      |
| OP_TIME   | Timestamp(6)  | Thời gian phát sinh của bản ghi tại nguồn.                                       |
| OP_NO     | number        | Số scn của hệ thống nguồn.                                                       |
| OP_RBA    | number        | Số thứ tự của trail file nối với số thứ tự bản ghi trong mỗi trail file trên OGG |
| LOAD_DATE | Timestamp (6) | Thời gian dữ liệu được tải vào vùng Landing.                                     |

- Partition: Các bảng landing được tạo partition theo ngày dựa trên cột OP\_TIME. Một số bảng đặc biệt có thể được partition theo các cột khác phù hợp với nhu cầu xử lý dữ liệu
- Data retention: Dữ liệu sẽ được áp dụng tự động xóa dữ liệu cũ hơn 30 ngày khỏi vùng Landing.

## 2. Chi tiết thiết kế hệ thống

### Phân vùng Data Vault – Giới thiệu Data Vault 2.0



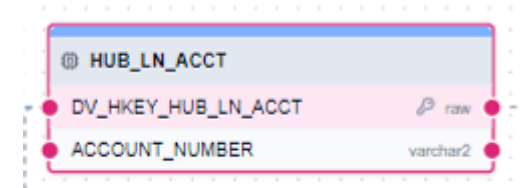
Tại Data Vault, các bảng (model) đều được tự động Create, Insert, Delete,... thông qua DBT (Data Build Tool).

## 2. Chi tiết thiết kế hệ thống

### Phân vùng Data Vault – Giới thiệu Data Vault 2.0

#### Hubs table (HUB)

- **Mục đích:**
  - Đại diện cho các thực thể kinh doanh cốt lõi (core business entities) hoặc các khái niệm duy nhất mà doanh nghiệp quan tâm.
- **Đặc điểm:**
  - Một Hub chỉ chứa các khóa kinh doanh (business keys) duy nhất và không thay đổi (ví dụ: Customer No, Account Number, Product Code).
  - Khóa kinh doanh này sẽ được mã hóa (hash) bằng thuật toán SHA-256 để tạo thành một giá trị băm duy nhất gọi là cột HKEY\_HUB.
  - Hub không chứa bất kỳ thuộc tính mô tả nào khác.



## 2. Chi tiết thiết kế hệ thống

### Phân vùng Data Vault – Giới thiệu Data Vault 2.0

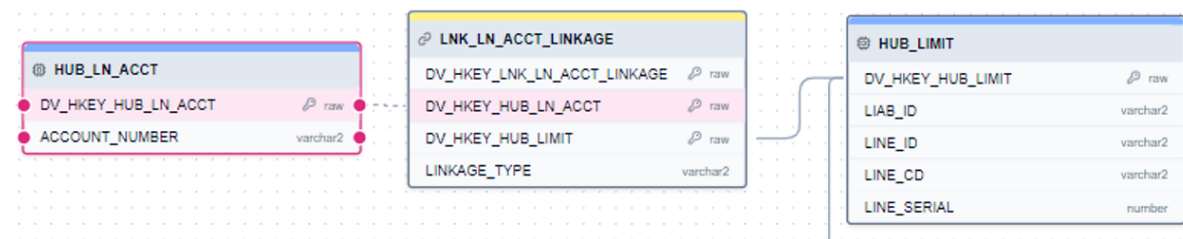
- **Links table (LINK)**

- **Mục đích:**

- Đại diện cho các mối quan hệ giữa hai hoặc nhiều thực thể (HUB)

- **Đặc điểm:**

- Mỗi Link sẽ có một khóa duy nhất và không thay đổi gọi là HKEY\_LINK. Giá trị này được tạo ra bằng cách mã hóa (hash) tổ hợp các khóa ngoại được mã hóa (HKEY\_HUB) trỏ đến các Hub liên quan.
- Một Link chứa các khóa ngoại được mã hóa (HKEY\_HUB) trỏ đến các Hub liên quan.
- Driven Key: Trong một số trường hợp, một hoặc nhiều HUB trong Link đóng vai trò xác định tính duy nhất của mối quan hệ tại 1 thời điểm. Các hub đó sẽ driven các hub còn lại, hashkey của hub đó sẽ được gọi là Driven Key. Ví dụ: trong link: LNK\_DP\_ACCT\_BRN\_CUST\_CCY\_ACLASS\_GL là quan hệ giữa account, chinh nhánh, khách hàng, sản phẩm và tài khoản kế toán. Thì tại một thời điểm 1 account chỉ có thể thuộc về 1 chi nhánh quản lý, được sở hữu bởi khách hàng và được gắn vào 1 đầu CR\_GL với 1 mã sản phẩm. Khác với link LNK\_DP\_INT\_PROD\_ACLASS\_CCY tại một thời điểm 1 ACLASS có thể có nhiều loại PRODUCT khác nhau.
  - Thông thường để xác định 1 link có xuất hiện driven key hay không ta sẽ xem xét tới primary key của bảng source. Nếu link có sự tham gia của các column không nằm trong bộ primary key của bảng source và các column đó lại có tham gia việc tạo thành 1 hub khác thì có thể xem xét xử lý driven key cho link đó.
  - Việc xét driven key mục đích để tối ưu hiệu suất( chi tiết giải thích ở Isate). Việc có hay không có driven key không ảnh hưởng tới việc logic lưu dữ liệu.
- Link không có thuộc tính mô tả nào về bản thân mối quan hệ đó.



## 2. Chi tiết thiết kế hệ thống

### Phân vùng Data Vault – Giới thiệu Data Vault 2.0

- **Satellites table (SAT)**
- **Mục đích:**
  - Lưu trữ các thuộc tính thay đổi theo thời gian của thực thể (Hub) hoặc mối quan hệ (Link). Trong datamodel các bảng satellite của hub được đặt tên bắt đầu với ký tự “SAT”, các bảng satellite của link được đặt tên bắt đầu với ký tự “LSAT”.
- **Đặc điểm:**
  - Mỗi bảng Satellite **chỉ liên kết với một bảng** Hub hoặc Link thông qua cột Hash Key.
  - Mỗi bảng Satellite có một cột Hash Diff được tạo ra bằng cách hash tất cả các cột trong bảng Sat trừ cột liên quan đến thuộc tính “KEY” và các thuộc tính hệ thống. Dùng để so sánh và đánh dấu sự thay đổi thông tin của dòng dữ liệu.
  - Satellites có thể có thêm các cột “depent key”. “Depent key” là thông thường là các cột tham gia vào constraint primary key của hệ thống source nhưng không phải là business key.

## 2. Chi tiết thiết kế hệ thống

### Phân vùng Data Vault – Giới thiệu Data Vault 2.0

#### Satellites table (SAT)

- **Đặc điểm:**
  - Chia theo cách lưu trữ và xử lý dữ liệu có các loại bảng satellites sau:
    - Sat/Lsat (main): Là các bảng lưu trữ toàn bộ lịch sử thay đổi các thuộc tính của hub hoặc link tương ứng. Trong phạm vi datamodel, đơn vị thời gian tính thay đổi là ngày. Nếu có thay đổi trên bất kì thuộc tính nào (trừ các cột system datavault) sẽ insert mới nếu không sẽ không insert. Cơ chế lưu trữ của nhóm bảng này là append only.
    - Sat\_der/Lsat\_der: Là các view lưu dòng dữ liệu mới nhất theo key của dữ liệu có phát sinh trong ngày cob\_date. Các view này được sinh ra tự động và tự làm mới metadata theo ngày thông qua dbt service (mỗi lần airflow call api của dbt sẽ sinh ra 1 bộ metadata mới của view).
    - Sat\_snp/Lsat\_snp: Là các bảng Sat lưu trữ dòng dữ liệu có hiệu lực mới nhất của hub hoặc link tại theo ngày xử lý dữ liệu.
      - Lưu phiên bản mới nhất của dữ liệu giúp tối ưu quá trình xử lý ETL
      - Cơ chế insert dữ liệu vào các bảng snapshot là cơ chế merge: Dữ liệu được tính toán lấy dòng mới nhất trong satellites der sau đó merge vào snapshot (không cần so sánh từng cột)
      - Các bảng snapshot được dung hầu hết trong các công đoạn ETL từ tính toán bizvault, fact.. Do đó phải luôn đảm bảo data chính xác cho nhóm bảng này.
  - Chia theo tính chất điều phối dữ liệu ta có 2 loại bảng satellites của link (tương ứng với các bảng link có và không có driven key):
    - Lsat thông thường:
    - Lsat effective:
      - Với mỗi quan hệ “một-một” tại một thời điểm, chỉ có duy nhất một mối quan hệ giữa các khóa. Sat effective date dùng để lưu trữ được ngày hiệu lực của mỗi quan hệ:

Danh sách các loại cột trong Raw vault

| Loại cột                     | Mô tả                                                                                                                                                                                                                                          | HUB | LNK | SAT | LSAT | LSATE |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|-----|-----|------|-------|
| Hash_key_hub                 | Đây là khóa băm (hash key) của một Hub, được tạo ra từ (các) khóa nghiệp vụ (business key). Nó đóng vai trò là khóa chính (primary key) của bảng Hub và là khóa ngoại (foreign key) trong các bảng Link và Satellite để tham chiếu đến Hub đó. | X   | X   | X   |      | X     |
| Hash_key_lnk                 | Là khóa băm của một Link, được tạo ra từ sự kết hợp của các business key của các Hub mà nó kết nối. Đây là khóa chính của bảng Link.                                                                                                           |     | X   |     | X    | X     |
| Hash_key_sat                 | Là khóa băm của một Satellite, được tạo ra từ (các) khóa nghiệp vụ và ngày giờ hiệu lực (load date). Nó và dependent_key (nếu có) là khóa chính của bảng Satellite.                                                                            |     |     | X   |      |       |
| Hash_key_lsate               | Là khóa băm của một Link Satellite, được tạo ra từ sự kết hợp của Hash_key_lnk và ngày giờ hiệu lực. Đây là khóa chính của bảng LSAT.                                                                                                          |     |     |     | X    |       |
| Hash_diff                    | Một giá trị băm được tính toán từ tất cả các thuộc tính mô tả trong một bảng Satellite. Nó được sử dụng để phát hiện sự thay đổi trong dữ liệu nguồn một cách hiệu quả.                                                                        |     |     | X   | X    |       |
| Biz_key                      | (Business Key) Là (các) mã định danh duy nhất cho một khái niệm nghiệp vụ trong hệ thống nguồn (ví dụ: mã khách hàng, mã sản phẩm). Đây là nền tảng để tạo ra Hash_key_hub.                                                                    | X   |     |     |      |       |
| Dependent_key                | (Dependent Key) Các cột khóa phụ thuộc trong bảng Satellite. Dùng để đánh dấu thuộc tính (Attr) này trong bảng satellite đang phụ thuộc vào một nghiệp vụ đặc biệt.                                                                            |     |     | X   | X    |       |
| Attr_column                  | (Attribute Column) Các cột chứa thuộc tính mô tả, mang thông tin chi tiết về khóa nghiệp vụ (trong Hub) hoặc mối quan hệ (trong Link). Đây là những dữ liệu có thể thay đổi theo thời gian.                                                    |     |     | X   | X    |       |
| System_columns<br>Dv_cdc_ops | Loại thao tác CDC (Change Data Capture) từ hệ thống nguồn (ví dụ: INIT, INSERT, UPDATE, DELETE). Cột này được lấy trực tiếp từ cột OP_TYPE trong bảng Landing.                                                                                 | X   | X   | X   | X    | X     |
| System_columns<br>Dv_src_ldt | Thời gian dữ liệu phát sinh tại nguồn. Cột này được lấy trực tiếp từ cột OP_TIME trong bảng Landing.                                                                                                                                           | X   | X   | X   | X    | X     |
| System_columns<br>Dv_scn     | System Change Number (SCN) tại thời điểm ghi nhận thay đổi từ nguồn. Cột này được lấy từ cột OP_NO trong bảng Landing.                                                                                                                         | X   | X   | X   | X    | X     |
| System_columns<br>Dv_rba     | Redo Byte Address (RBA) trong redo log của hệ thống nguồn. Thông tin được lấy từ cột OP_RBA trong bảng Landing.                                                                                                                                | X   | X   | X   | X    | X     |
| System_columns<br>Dv_src_rec | Thông tin lưu trữ tên bảng Landing của dữ liệu.                                                                                                                                                                                                | X   | X   | X   | X    | X     |
| System_columns<br>Dv_ldt     | Load Date Timestamp. Thời gian dữ liệu được tải vào bảng Rawvault.                                                                                                                                                                             | X   | X   | X   | X    | X     |
| System_columns<br>DV_CCD     | Cột này chỉ tồn tại trong HUB. Collision code, default: 'NAB'.                                                                                                                                                                                 | X   |     |     |      |       |



## 2. Chi tiết thiết kế hệ thống

### Phân vùng Data Vault – Giới thiệu Data Vault 2.0

#### Reference Table (Ref)

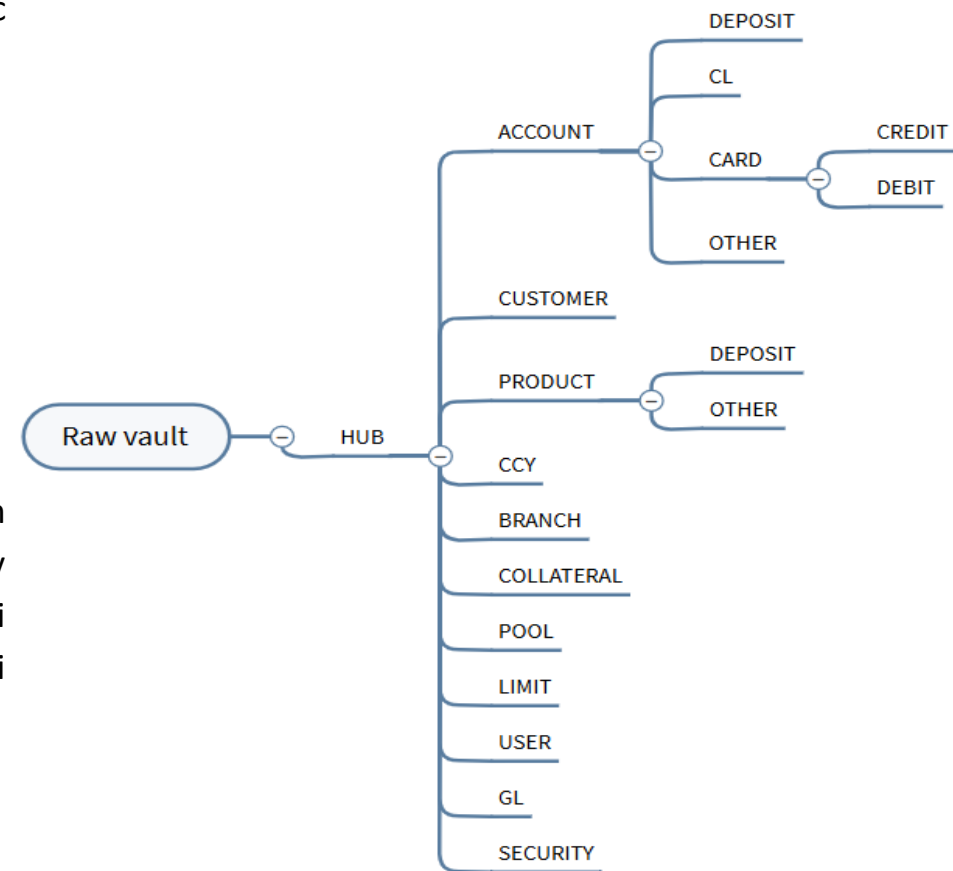
- Mục đích: Lưu trữ các bảng tham chiếu (các bảng định nghĩa danh mục, các bảng system) hoặc các bảng không thiết kế trực tiếp hub, sat, link.
- Đặc điểm:
  - Tên bảng ref được tạo ra bằng cách thêm “ref\_” + tên table landing.
  - Trong ref table, không xuất hiện hash column, chỉ có các cột hệ thống của datavault( dv\_src\_ldt, dv\_scn, dv\_rba, dv\_ldt)
  - Cơ chế lưu trữ dữ liệu trong ref table là lấy dòng dữ liệu phát sinh mới nhất theo từng ngày xử lý dữ liệu.
  - Chia theo cách sử dụng dữ liệu có 2 loại bảng ref:
    - Ref master: Là nhóm bảng khi muốn lấy được dữ liệu tương ứng với ngày xử lý cob\_date phải sử dụng bộ 3 ( op\_time, op\_no, op\_rba) trong bảng landing tương ứng để xác định dòng dữ liệu mới nhất tính tới ngày cob\_date.
    - ```
Select * from (Select 'key', 'attribute', row_number() over(partition by key order by dv_src_ldt desc, dv_scn desc, dv_rba desc) rn Where cob_date <= p_date) where rn=1
```
    - Ref Transaction: Khác với ref master, nhóm bảng này chỉ cần lọc điều kiện ngày cob\_date= p\_date là có thể lấy được dữ liệu phát sinh tại ngày xử lý dữ liệu.

## 2. Chi tiết thiết kế hệ thống Tổ chức Raw Vault trong Data Model

- Về tổ chức phân vùng rawvault trong datamodel. Dữ liệu được lấy từ các nguồn các:

- Hệ thống corebanking( Flexlive)
- Hệ thống tổng hợp dữ liệu sử dụng cho báo cáo (Flexbo)
- Hệ thống xử lý cash(Flexcash)
- Hệ thống datawarehouse(flexdw)
- Hệ thống xử lý cân đối kế toán (converter01)

- Hệ thống core là hệ thống trung tâm, các business chính được phân tích trên datamodel đều được tổng hợp từ flexlive. Flexbo được sử dụng để lấy data 2 phân hệ chính là Thẻ ( debit, credit) và thông tin GL, các bảng còn lại trong flexbo chủ yếu sử dụng làm reference table. Tương tự 2 hệ thống còn lại Flexcard và NabDW chủ yếu làm thông tin tham chiếu.



## 2. Chi tiết thiết kế hệ thống

### Phân vùng Biz Vault

#### **Logic Chuyển đổi trong Biz Vault**

Các logic chuyển đổi trong Biz Vault được thiết kế để phục vụ các yêu cầu cụ thể của nghiệp vụ:

- Chuẩn hóa và làm sạch dữ liệu nghiệp vụ: Tại Biz Vault có thể áp dụng các quy tắc làm sạch hoặc chuẩn hóa bổ sung dựa trên định nghĩa nghiệp vụ (ví dụ: chuyển đổi đơn vị, chuẩn hóa tên địa lý theo danh mục nội bộ).
- Tổng hợp và tính toán các chỉ số nghiệp vụ: Tạo ra các chỉ số (KPIs) hoặc thuộc tính phái sinh (derived attributes) quan trọng cho nghiệp vụ (ví dụ: tổng dự thu trong kỳ của từng tài khoản, số lần giao dịch của khách hàng,...).

#### **Các thành phần chính của Biz Vault**

Biz Vault bao gồm hai loại bảng chính để thực hiện các chức năng trên:

- Bridge Tables
- Business Satellite Tables (sat\_biz).

## 2. Chi tiết thiết kế hệ thống

### Phân vùng Biz Vault - Bridge Tables

**Bridge Tables** trong Biz Vault không giống hoàn toàn với các bảng liên kết (link tables) trong Raw Vault. Thay vào đó, chúng được sử dụng để:

- **Mô hình hóa các mối quan hệ nghiệp vụ phức tạp hoặc đa chiều:** Khi có nhiều hơn hai Hub hoặc Link tham gia vào một mối quan hệ mà nghiệp vụ cần nhìn nhận dưới một góc độ tổng hợp.
- **Tạo ra các mối quan hệ "cầu nối" hiệu quả hơn cho truy vấn:** Thay vì phải join qua nhiều Hub và Link trong Raw Vault để trả lời một câu hỏi nghiệp vụ, Bridge Table có thể pre-join các Hub/Link đó lại, giúp tăng hiệu suất truy vấn cho Data Mart.
- **Tạo ra các tập hợp dữ liệu con (subsets) theo nghiệp vụ:** Ví dụ, một Bridge Table nhóm các tài khoản có Transfer Limit,...

